📘 Guía Maestra Definitiva: Examen CRUD Full-Stack (.NET 10 + Angular)

> **Manual de Estudio y Plantilla Paso a Paso para Evaluaciones Prácticas**
> *Ingeniería de Sistemas - UNJFSC | Asignatura: Desarrollo de Sistemas de Información (DSI)*
> *Patrón: Dashboard Administrativo Maestro-Detalle con Barra Lateral (Sidebar)*

📑 Tabla de Contenidos

1. La Lógica y Conceptos: ¿Para Qué Sirve Cada Archivo?
2. El Flujo de Datos Completo (Del Clic a la Base de Datos)
3. Parte 1: Backend (.NET Web API Simple y Completo)
4. 3.1 Modelos C# (`DetalleOperacion.cs` y `CabeceraOperacion.cs`)
5. 3.2 Contexto de Datos (`ApplicationDbContext.cs`)
6. 3.3 Capa de Acceso a Datos (`DADetalleEjecuciones.cs`)
7. 3.4 Controlador API REST (`BackEjecucionesController.cs`)
8. 3.5 Configuración de CORS en `Program.cs`
9. Parte 2: Frontend (Angular Paso a Paso desde Cero)
10. 4.1 Comandos de Creación en Terminal
11. 4.2 Configuración Obligatoria de `provideHttpClient` en `app.config.ts`
12. 4.3 Modelos TypeScript (`detalle.model.ts` y `cabecera.model.ts`)
13. 4.4 Servicio HTTP (`ejecuciones.services.ts`)
14. 4.5 Lógica TypeScript del Componente (`panel.ts`)
15. 4.6 Vista HTML de Clase con Sidebar, Tabla y Modales (`panel.html`)
16. 4.7 Hoja de Estilos del Dashboard (`panel.css`)
17. Parte 3: Cronograma Estratégico de 90 Minutos para el Examen
18. Parte 4: Top 5 Errores Típicos de Examen y Solución en 5 Segundos

---

1. La Lógica y Conceptos: ¿Para Qué Sirve Cada Archivo?

Para no memorizar a ciegas, imagina que una aplicación Web Full-Stack es un **Restaurante de Comida**:

```
[ FRONTEND (Angular) ]                    [ BACKEND (.NET Web API) ]              [ BASE DE DATOS
(SQL Server) ]
   • Cliente sentado en la mesa              • El Mozo y la Cocina                   • El Almacén /
Despensa
   • Ve la carta (HTML)                      • Controller: atiende el pedido         • Guarda los
registros físicos
   • Hace clic en un botón                   • DA: saca los ingredientes             • Tablas:
   • Service: lleva la orden                 • DbContext: abre la puerta del almacén   -
CabeceraOperacion
                                                                                       -
DetalleOperacion
```

¿Qué hace cada archivo en el Frontend (Angular)?

1. `app.config.ts` : Es el **interruptor general**. Por defecto, Angular viene cerrado al exterior. Con `provideHttpClient()` le das permiso a la aplicación para hacer llamadas HTTP (GET, POST, etc.) a servidores externos.
2. `models/detalle.model.ts` : Es el **contrato o molde**. Define la estructura de los datos (qué campos existen: `idDetalleOperacion` , `descripcion` , `cantidad` , `importe` , etc.). Si escribes mal un campo, TypeScript te avisa de inmediato.
3. `services/ejecuciones.services.ts` : Es el **mensajero**. No dibuja nada en pantalla. Su único trabajo es comunicarse con el Backend mediante `HttpClient` y devolver un `Observable` con la respuesta.
4. `components/panel/panel.ts` **(El Cerebro)**: Guarda el estado de la vista en variables (`cargando` , `mostrarTabla` , `sidebarContraido` , `detalleEjecucion`) y define funciones para responder a los eventos del usuario (`cargarDetalleEjecucion()` , `guardar()` , `eliminar()`).
5. `components/panel/panel.html` **(La Vista)**: El código visual que el usuario ve y toca.
6. `components/panel/panel.css` **(El Estilo)**: Define los colores, la barra lateral oscura, la tabla limpia y los modales flotantes.

Sintaxis Clave de Angular en el HTML:

- `{{ variable }}` **(Interpolación):** Muestra el valor de una variable de TypeScript en la pantalla.
- `[propiedad]="variable"` **(Property Binding):** Modifica una propiedad HTML según el valor de TypeScript (ej: `[class.active]="mostrarTabla"`).
- `(click)="metodo()"` **(Event Binding):** Dispara una función de TypeScript al hacer clic.
- `[(ngModel)]="formulario.campo"` **(Two-Way Binding):** Sincronización bidireccional. Lo que el usuario escribe en la caja de texto actualiza la variable en tiempo real.
- `*ngIf="condicion"` **:** Muestra u oculta elementos según si la condición es verdadera o falsa.
- `*ngFor="let item of lista"` **:** Itera sobre un arreglo y genera una fila por cada registro.

---

2. El Flujo de Datos Completo (Del Clic a la Base de Datos)

1. El usuario hace clic en el menú **"Ejecuciones"** `(click)="cargarDetalleEjecucion()"` .
2. `panel.ts` pone `cargando = true` y llama a `this.ejecucionesService.getEjecuciones()` .
3. El servicio dispara un `HTTP GET http://localhost:5264/api/ejecuciones` .
4. El Backend recibe la petición en `BackEjecucionesController.cs` con el atributo `[HttpGet]` .
5. El controlador llama a `_da.GetDetalleOperacion()` en `DADetalleEjecuciones.cs` .
6. El `DA` usa `ApplicationDbContext` para consultar la tabla en SQL Server y retorna una lista `List<DetalleOperacion>` .
7. El controlador responde con un código HTTP 200 y el JSON con los datos.
8. En Angular, el método `.subscribe({ next: (datos) => ... })` recibe el JSON.
9. Guardamos los datos en `this.detalleEjecucion = datos` , ponemos `cargando = false` y `mostrarTabla = true` .
10. El HTML detecta el cambio automáticamente y el `*ngFor` dibuja cada fila en la tabla.

---

3. Parte 1: Backend (.NET Web API Simple y Completo)

3.1 Modelos C# (`Models/DetalleOperacion.cs` y `Models/CabeceraOperacion.cs`)

```
// Models/CabeceraOperacion.cs
using System.ComponentModel.DataAnnotations;

namespace BackendWebApp.Models
{
    public class CabeceraOperacion
    {
        [Key]
        public int IdCabeceraOperacion { get; set; }

        [Required]
        [MaxLength(100)]
        public string Cliente { get; set; } = string.Empty;

        public DateTime Registro { get; set; } = DateTime.Now;
    }
}
```

```
// Models/DetalleOperacion.cs
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace BackendWebApp.Models
{
    public class DetalleOperacion
    {
        [Key]
        public int IdDetalleOperacion { get; set; }

        [Required]
        [MaxLength(100)]
        public string Descripcion { get; set; } = string.Empty;

        [Required]
        public int Cantidad { get; set; }

        [Required]
        public float Importe { get; set; }

        public float Igv { get; set; }

        public float Total { get; set; }

        // Relación con Cabecera
        public int IdCabeceraOperacion { get; set; }

        [ForeignKey("IdCabeceraOperacion")]
        public virtual CabeceraOperacion? CabeceraOperacion { get; set; }
    }
}
```

3.2 Contexto de Datos (`Data/ApplicationDbContext.cs`)

```
using BackendWebApp.Models;
using Microsoft.EntityFrameworkCore;

namespace BackendWebApp.Data
{
    public class ApplicationDbContext : DbContext
    {
        public DbSet<CabeceraOperacion> CabeceraOperacion { get; set; }
        public DbSet<DetalleOperacion> DetalleOperacion { get; set; }

        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            if (!optionsBuilder.IsConfigured)
            {
                // Ajusta el nombre de tu servidor SQL si es diferente

optionsBuilder.UseSqlServer("Server=.;Database=BD_Ejecuciones;Trusted_Connection=True;TrustServerCert
ificate=True;");
            }
        }
    }
}
```

3.3 Capa de Acceso a Datos (`Data/AccesoDatos/DADetalleEjecuciones.cs`)

Métodos directos, limpios y robustos con bloques `using`:

```
using BackendWebApp.Models;
using Microsoft.EntityFrameworkCore;
using System.Collections.Generic;
using System.Linq;

namespace BackendWebApp.Data.AccesoDatos
{
    public class DADetalleEjecuciones
    {
        // 1. LISTAR TODOS
        public IEnumerable<DetalleOperacion> GetDetalleOperacion()
        {
            using (var db = new ApplicationDbContext())
            {
                return db.DetalleOperacion.Include(x => x.CabeceraOperacion).ToList();
            }
        }

        // 2. OBTENER POR ID
        public DetalleOperacion? GetDetalleOperacionById(int id)
        {
            using (var db = new ApplicationDbContext())
            {
                return db.DetalleOperacion.Include(x => x.CabeceraOperacion)
                                          .FirstOrDefault(x => x.IdDetalleOperacion == id);
            }
        }

        // 3. INSERTAR REGISTRO
        public int InsertDetalleOperacion(DetalleOperacion entidad)
        {
            using (var db = new ApplicationDbContext())
            {
                entidad.CabeceraOperacion = null; // Evita conflicto de entidad foránea
                db.DetalleOperacion.Add(entidad);
                db.SaveChanges();
                return entidad.IdDetalleOperacion;
            }
        }

        // 4. ACTUALIZAR REGISTRO
        public bool UpdateDetalleOperacion(DetalleOperacion entidad)
        {
            using (var db = new ApplicationDbContext())
            {
                var existente = db.DetalleOperacion.Find(entidad.IdDetalleOperacion);
                if (existente == null) return false;

                existente.Descripcion = entidad.Descripcion;
                existente.Cantidad = entidad.Cantidad;
                existente.Importe = entidad.Importe;
                existente.Igv = entidad.Igv;
                existente.Total = entidad.Total;
                existente.IdCabeceraOperacion = entidad.IdCabeceraOperacion;

                db.SaveChanges();
                return true;
            }
        }
```

```
        // 5. ELIMINAR REGISTRO
        public bool DeleteDetalleOperacion(int id)
        {
            using (var db = new ApplicationDbContext())
            {
                var reg = db.DetalleOperacion.Find(id);
                if (reg == null) return false;

                db.DetalleOperacion.Remove(reg);
                db.SaveChanges();
                return true;
            }
        }
    }
}
```

3.4 Controlador API REST (`Controllers/BackEjecucionesController.cs`)

```
using BackendWebApp.Data.AccesoDatos;
using BackendWebApp.Models;
using Microsoft.AspNetCore.Mvc;

namespace BackendWebApp.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    [Route("api/ejecuciones")]
    public class BackEjecucionesController : ControllerBase
    {
        private readonly DADetalleEjecuciones _da = new DADetalleEjecuciones();

        [HttpGet]
        public IActionResult Listar() => Ok(_da.GetDetalleOperacion());

        [HttpGet("{id}")]
        public IActionResult ObtenerPorId(int id)
        {
            var item = _da.GetDetalleOperacionById(id);
            return item == null ? NotFound() : Ok(item);
        }

        [HttpPost]
        public IActionResult Crear([FromBody] DetalleOperacion entidad)
        {
            entidad.IdDetalleOperacion = 0;
            entidad.Igv = (float)Math.Round(entidad.Importe * 0.18f, 2);
            entidad.Total = (float)Math.Round(entidad.Importe + entidad.Igv, 2);

            var id = _da.InsertDetalleOperacion(entidad);
            if (id <= 0) return BadRequest("Error al insertar");

            return Ok(new { mensaje = "Registro creado con éxito", id });
        }

        [HttpPut("{id}")]
        public IActionResult Actualizar(int id, [FromBody] DetalleOperacion entidad)
        {
            entidad.IdDetalleOperacion = id;
            entidad.Igv = (float)Math.Round(entidad.Importe * 0.18f, 2);
            entidad.Total = (float)Math.Round(entidad.Importe + entidad.Igv, 2);

            var ok = _da.UpdateDetalleOperacion(entidad);
            return ok ? Ok(new { mensaje = "Actualizado" }) : NotFound();
        }

        [HttpDelete("{id}")]
        public IActionResult Eliminar(int id)
        {
            var ok = _da.DeleteDetalleOperacion(id);
            return ok ? Ok(new { mensaje = "Eliminado" }) : NotFound();
        }
    }
}
```

3.5 Configuración de CORS en `Program.cs` ⚠️ (OBLIGATORIO)

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// 1. POLÍTICA DE CORS PERMISIVA PARA ANGULAR
builder.Services.AddCors(options =>
{
    options.AddPolicy("PermitirTodo", policy =>
    {
        policy.AllowAnyOrigin()
              .AllowAnyMethod()
              .AllowAnyHeader();
    });
});

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

// 2. ACTIVAR CORS ANTES DE MAPEAR CONTROLADORES
app.UseCors("PermitirTodo");

app.UseAuthorization();
app.MapControllers();
app.Run();
```

---

4. Parte 2: Frontend (Angular Paso a Paso desde Cero)

4.1 Comandos de Creación en Terminal

```
# 1. Crear el proyecto Angular Standalone
ng new frontend-angular --routing --style=css
cd frontend-angular

# 2. Generar el componente del panel
ng g c components/panel --skip-tests

# 3. Generar el servicio de comunicaciones HTTP
ng g s services/ejecuciones --skip-tests
```

4.2 Configuración Obligatoria en `src/app/app.config.ts`

```
import { ApplicationConfig, provideZoneChangeDetection } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient } from '@angular/common/http'; // 👈 1. IMPORTAR
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    provideHttpClient() // 👈 2. ACTIVAR EL CLIENTE HTTP
  ]
};
```

4.3 Modelos TypeScript (`src/app/models/detalle.model.ts`)

```
export interface DetalleOperacion {
  idDetalleOperacion?: number;
  descripcion: string;
  cantidad: number;
  importe: number;
  igv?: number;
  total?: number;
  idCabeceraOperacion: number;
}
```

4.4 Servicio HTTP (`src/app/services/ejecuciones.services.ts`)

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { DetalleOperacion } from '../models/detalle.model';

@Injectable({
  providedIn: 'root'
})
export class EjecucionesService {
  private http = inject(HttpClient);
  private apiUrl = 'http://localhost:5264/api/ejecuciones';

  // GET: Listar todos
  getEjecuciones(): Observable<DetalleOperacion[]> {
    return this.http.get<DetalleOperacion[]>(this.apiUrl);
  }

  // GET: Obtener uno por ID
  getEjecucionById(id: number): Observable<DetalleOperacion> {
    return this.http.get<DetalleOperacion>(`${this.apiUrl}/${id}`);
  }

  // POST: Crear nuevo
  crearEjecucion(item: DetalleOperacion): Observable<DetalleOperacion> {
    return this.http.post<DetalleOperacion>(this.apiUrl, item);
  }

  // PUT: Actualizar existente
  actualizarEjecucion(id: number, item: DetalleOperacion): Observable<any> {
    return this.http.put<any>(`${this.apiUrl}/${id}`, item);
  }

  // DELETE: Eliminar por ID
  eliminarEjecucion(id: number): Observable<any> {
    return this.http.delete<any>(`${this.apiUrl}/${id}`);
  }
}
```

4.5 Lógica TypeScript del Componente (`src/app/components/panel/panel.ts`)

```
import { Component, OnInit, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { EjecucionesService } from '../../services/ejecuciones.services';
import { DetalleOperacion } from '../../models/detalle.model';

@Component({
  selector: 'app-panel',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './panel.html',
  styleUrl: './panel.css'
})
export class PanelComponent implements OnInit {
  // 1. Inyección del servicio
  private ejecucionesService = inject(EjecucionesService);

  // 2. Control de la interfaz (UI)
  sidebarContraido: boolean = false;
  dashboardAbierto: boolean = true;
  usuario: string = 'Marco Torres (UNJFSC)';

  // 3. Estados de la vista
  cargando: boolean = false;
  mostrarTabla: boolean = false;
  error: boolean = false;

  // 4. Datos en memoria
  detalleEjecucion: DetalleOperacion[] = [];

  // 5. Control de modales y formularios
  mostrarModalFormulario: boolean = false;
  modoEdicion: boolean = false;
  mostrarModalDetalle: boolean = false;
  itemSeleccionado: DetalleOperacion | null = null;

  formulario: DetalleOperacion = this.limpiarFormulario();

  ngOnInit(): void {}

  toggleSidebar(): void {
    this.sidebarContraido = !this.sidebarContraido;
  }

  toggleDashboard(): void {
    this.dashboardAbierto = !this.dashboardAbierto;
  }

  // Cargar datos vía GET
  cargarDetalleEjecucion(): void {
    this.cargando = true;
    this.error = false;
    this.mostrarTabla = false;

    this.ejecucionesService.getEjecuciones().subscribe({
      next: (datos) => {
        this.detalleEjecucion = datos;
        this.cargando = false;
```

```
        this.mostrarTabla = true;
      },
      error: (err) => {
        console.error('Error al conectar:', err);
        this.cargando = false;
        this.error = true;
      }
    });
  }

  // Ver detalle
  verDetalle(id?: number): void {
    if (!id) return;
    const encontrado = this.detalleEjecucion.find(x => x.idDetalleOperacion === id);
    if (encontrado) {
      this.itemSeleccionado = encontrado;
      this.mostrarModalDetalle = true;
    }
  }

  // Abrir modal de creación
  abrirModalCrear(): void {
    this.modoEdicion = false;
    this.formulario = this.limpiarFormulario();
    this.mostrarModalFormulario = true;
  }

  // Abrir modal de edición
  abrirModalEditar(item: DetalleOperacion): void {
    this.modoEdicion = true;
    this.formulario = { ...item };
    this.mostrarModalFormulario = true;
  }

  // Cálculo en vivo de IGV y Total
  calcularTotales(): void {
    const cant = this.formulario.cantidad || 0;
    const imp = this.formulario.importe || 0;
    const subtotal = cant * imp;
    this.formulario.igv = parseFloat((subtotal * 0.18).toFixed(2));
    this.formulario.total = parseFloat((subtotal + this.formulario.igv).toFixed(2));
  }

  // Guardar (POST o PUT)
  guardar(): void {
    this.calcularTotales();

    if (this.modoEdicion && this.formulario.idDetalleOperacion) {
      this.ejecucionesService.actualizarEjecucion(this.formulario.idDetalleOperacion,
this.formulario).subscribe({
        next: () => {
          alert('¡Ejecución actualizada con éxito!');
          this.cerrarModalFormulario();
          this.cargarDetalleEjecucion();
        },
        error: () => alert('Error al actualizar el registro.')
      });
    } else {
      this.ejecucionesService.crearEjecucion(this.formulario).subscribe({
        next: () => {
          alert('¡Ejecución creada con éxito!');
```

```
          this.cerrarModalFormulario();
          this.cargarDetalleEjecucion();
        },
        error: () => alert('Error al crear el registro.')
      });
    }
  }

  // Eliminar (DELETE)
  eliminar(id?: number): void {
    if (!id) return;
    if (!confirm(`¿Estás seguro de eliminar el registro #${id}?`)) return;

    this.ejecucionesService.eliminarEjecucion(id).subscribe({
      next: () => {
        alert('Registro eliminado.');
        this.cargarDetalleEjecucion();
      },
      error: () => alert('No se pudo eliminar el registro.')
    });
  }

  cerrarModalFormulario(): void {
    this.mostrarModalFormulario = false;
    this.formulario = this.limpiarFormulario();
  }

  private limpiarFormulario(): DetalleOperacion {
    return {
      idDetalleOperacion: 0,
      descripcion: '',
      cantidad: 1,
      importe: 0,
      igv: 0,
      total: 0,
      idCabeceraOperacion: 1
    };
  }
}
```

4.6 Vista HTML de Clase (`src/app/components/panel/panel.html`)

```
<div class="admin-layout" [class.sidebar-mini]="sidebarContraido">

  <!-- 1. BARRA LATERAL (SIDEBAR) -->
  <aside class="sidebar">
    <div class="brand">
      <div class="brand-icon">A</div>
      <div class="brand-text" *ngIf="!sidebarContraido">AngularDSI</div>
    </div>

    <div class="sidebar-content">
      <div class="menu-block">

        <div class="menu-item menu-item-main" (click)="toggleDashboard()">
          <div class="menu-left">
            <span class="menu-icon">📊</span>
            <span class="menu-label" *ngIf="!sidebarContraido">DashBoard</span>
          </div>
          <span class="menu-arrow" *ngIf="!sidebarContraido">
            {{ dashboardAbierto ? '▼' : '►' }}
          </span>
        </div>

        <div class="submenu" *ngIf="dashboardAbierto && !sidebarContraido">
          <div class="submenu-item" [class.active]="mostrarTabla" (click)="cargarDetalleEjecucion()">
            <span class="submenu-icon">📑</span>
            <span class="submenu-label">Ejecuciones</span>
          </div>

          <div class="submenu-item">
            <span class="submenu-icon">📈</span>
            <span class="submenu-label">DashBoard 1</span>
          </div>

          <div class="submenu-item">
            <span class="submenu-icon">📉</span>
            <span class="submenu-label">DashBoard 2</span>
          </div>
        </div>

      </div>
    </div>
  </aside>

  <!-- 2. CONTENIDO PRINCIPAL -->
  <div class="main-wrapper">

    <header class="topbar">
      <div class="topbar-left">
        <button class="menu-toggle" (click)="toggleSidebar()" title="Colapsar menú">☰</button>
        <a href="#" class="top-link">Home</a>
        <a href="#" class="top-link">Contact</a>
      </div>
      <div class="topbar-right">
        <div class="user-info">
          <div class="avatar">{{ usuario.charAt(0) }}</div>
          <span class="user-name">{{ usuario }}</span>
        </div>
      </div>
```

```
    </header>

    <main class="content-area">

      <!-- Estado: Inicial -->
      <div class="estado-box" *ngIf="!cargando && !mostrarTabla && !error">
        <div class="estado-card">
          <i class="bi bi-info-circle fs-1 text-primary mb-2"></i>
          <h2>Seleccione una opción del menú</h2>
          <p class="text-muted">Haga clic en <strong>DashBoard ➔ Ejecuciones</strong> en el menú
lateral para consultar los datos.</p>
        </div>
      </div>

      <!-- Estado: Cargando -->
      <div class="estado-box" *ngIf="cargando">
        <div class="estado-card">
          <div class="spinner-border text-primary mb-3" role="status"></div>
          <h2>Cargando la información...</h2>
          <p class="text-muted">Conectando con la API en .NET...</p>
        </div>
      </div>

      <!-- Estado: Error -->
      <div class="estado-box" *ngIf="error && !cargando">
        <div class="estado-card border-danger">
          <i class="bi bi-exclamation-triangle-fill fs-1 text-danger mb-2"></i>
          <h2 class="text-danger">Error de Conexión</h2>
          <p class="text-muted">No se pudo conectar con el Backend. Verifica que esté ejecutándose en
el puerto 5264.</p>
          <button class="btn btn-outline-danger mt-2"
(click)="cargarDetalleEjecucion()">Reintentar</button>
        </div>
      </div>

      <!-- Estado: Tabla con Datos -->
      <div class="tabla-seccion" *ngIf="mostrarTabla && !cargando">

        <div class="toolbar">
          <div>
            <h3 class="m-0 fw-bold">Listado de Ejecuciones Presupuestales</h3>
            <small class="text-muted">Total de registros: {{ detalleEjecucion.length }}</small>
          </div>
          <button class="btn-nuevo" (click)="abrirModalCrear()">
            <i class="bi bi-plus-circle me-1"></i> Nueva Ejecución
          </button>
        </div>

        <div class="tabla-container">
          <table class="tabla" *ngIf="detalleEjecucion.length > 0">
            <thead>
              <tr>
                <th class="text-center">Id</th>
                <th>Descripción</th>
                <th class="text-center">Cantidad</th>
                <th class="text-end">Importe</th>
                <th class="text-end">IGV (18%)</th>
                <th class="text-end">Total</th>
                <th class="text-center">Opciones</th>
              </tr>
            </thead>
```

```
            <tbody>
              <tr *ngFor="let item of detalleEjecucion">
                <td class="text-center fw-bold">{{ item.idDetalleOperacion }}</td>
                <td>{{ item.descripcion }}</td>
                <td class="text-center">{{ item.cantidad }}</td>
                <td class="text-end">S/. {{ item.importe | number:'1.2-2' }}</td>
                <td class="text-end text-muted">S/. {{ item.igv | number:'1.2-2' }}</td>
                <td class="text-end fw-bold text-success">S/. {{ item.total | number:'1.2-2' }}</td>
                <td class="text-center">
                  <button class="btn-accion btn-editar" (click)="abrirModalEditar(item)"
title="Editar">
                    <i class="bi bi-pencil-fill"></i>
                  </button>
                  <button class="btn-accion btn-detalle"
(click)="verDetalle(item.idDetalleOperacion)" title="Ver Detalle">
                    <i class="bi bi-eye-fill"></i>
                  </button>
                  <button class="btn-accion btn-eliminar" (click)="eliminar(item.idDetalleOperacion)"
title="Eliminar">
                    <i class="bi bi-trash-fill"></i>
                  </button>
                </td>
              </tr>
            </tbody>
          </table>

          <div *ngIf="detalleEjecucion.length === 0" class="p-4 text-center text-muted">
            No se encontraron registros de ejecuciones.
          </div>
        </div>
      </div>

    </main>

    <footer class="footer">
      <span>Copyright @2026 <strong>Ingeniería de Sistemas - UNJFSC</strong></span>
    </footer>
  </div>

  <!-- MODAL: CREAR / EDITAR -->
  <div class="modal-overlay" *ngIf="mostrarModalFormulario">
    <div class="modal-caja">
      <div class="modal-header">
        <h5>{{ modoEdicion ? '✏️ Editar Ejecución' : '➕ Nueva Ejecución' }}</h5>
        <button class="btn-cerrar" (click)="cerrarModalFormulario()">✕</button>
      </div>

      <div class="modal-body">
        <form (ngSubmit)="guardar()">

          <div class="form-grupo">
            <label>Descripción:</label>
            <input type="text" [(ngModel)]="formulario.descripcion" name="descripcion" required
class="form-input">
          </div>

          <div class="form-fila">
            <div class="form-grupo">
              <label>Cantidad:</label>
              <input type="number" [(ngModel)]="formulario.cantidad" name="cantidad" required min="1"
(input)="calcularTotales()" class="form-input">
```

```
            </div>

            <div class="form-grupo">
              <label>Importe Unitario / Base:</label>
              <input type="number" [(ngModel)]="formulario.importe" name="importe" required min="0"
step="0.01" (input)="calcularTotales()" class="form-input">
            </div>
          </div>

          <div class="calculos-box">
            <div><span>IGV (18%):</span> <strong>S/. {{ formulario.igv | number:'1.2-2' }}</strong>
</div>
            <div><span>Total:</span> <strong class="text-success">S/. {{ formulario.total |
number:'1.2-2' }}</strong></div>
          </div>

          <div class="form-grupo mt-2">
            <label>Código de Cabecera:</label>
            <input type="number" [(ngModel)]="formulario.idCabeceraOperacion" name="idCabecera"
class="form-input">
          </div>

          <div class="modal-footer">
            <button type="button" class="btn btn-secondary"
(click)="cerrarModalFormulario()">Cancelar</button>
            <button type="submit" class="btn btn-primary">
              <i class="bi bi-check-lg me-1"></i> {{ modoEdicion ? 'Actualizar' : 'Guardar' }}
            </button>
          </div>

        </form>
      </div>
    </div>
  </div>

  <!-- MODAL: VER DETALLE -->
  <div class="modal-overlay" *ngIf="mostrarModalDetalle && itemSeleccionado">
    <div class="modal-caja">
      <div class="modal-header">
        <h5>👁 Detalle de la Operación #{{ itemSeleccionado.idDetalleOperacion }}</h5>
        <button class="btn-cerrar" (click)="mostrarModalDetalle = false">✕</button>
      </div>
      <div class="modal-body">
        <ul class="detalle-lista">
          <li><strong>ID:</strong> {{ itemSeleccionado.idDetalleOperacion }}</li>
          <li><strong>Descripción:</strong> {{ itemSeleccionado.descripcion }}</li>
          <li><strong>Cantidad:</strong> {{ itemSeleccionado.cantidad }} unidades</li>
          <li><strong>Importe Base:</strong> S/. {{ itemSeleccionado.importe | number:'1.2-2' }}</li>
          <li><strong>IGV (18%):</strong> S/. {{ itemSeleccionado.igv | number:'1.2-2' }}</li>
          <li><strong>Total a Pagar:</strong> <span class="badge-total">S/. {{ itemSeleccionado.total
| number:'1.2-2' }}</span></li>
          <li *ngIf="itemSeleccionado.idCabeceraOperacion"><strong>Cabecera ID:</strong> {{
itemSeleccionado.idCabeceraOperacion }}</li>
        </ul>
      </div>
      <div class="modal-footer">
        <button class="btn btn-secondary" (click)="mostrarModalDetalle = false">Cerrar</button>
      </div>
    </div>
  </div>
```

```
</div>
```

4.7 Hoja de Estilos del Dashboard (`src/app/components/panel/panel.css`)

```
.admin-layout {
  display: flex;
  min-height: 100vh;
  font-family: 'Segoe UI', Roboto, sans-serif;
  background-color: #f4f6f9;
  color: #333;
}

.sidebar {
  width: 250px;
  background-color: #343a40;
  color: #c2c7d0;
  display: flex;
  flex-direction: column;
  transition: width 0.3s ease;
  user-select: none;
  flex-shrink: 0;
}

.brand {
  display: flex;
  align-items: center;
  gap: 12px;
  padding: 15px 20px;
  background-color: #2c3136;
  border-bottom: 1px solid #4b545c;
  color: #fff;
  font-size: 1.15rem;
  font-weight: bold;
}

.brand-icon {
  width: 32px;
  height: 32px;
  background-color: #007bff;
  color: #fff;
  border-radius: 6px;
  display: flex;
  align-items: center;
  justify-content: center;
  font-weight: 800;
}

.sidebar-content {
  padding: 12px 8px;
}

.menu-item {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 10px 14px;
  border-radius: 6px;
  cursor: pointer;
  color: #c2c7d0;
}

.menu-item:hover {
```

```
  background-color: rgba(255, 255, 255, 0.1);
  color: #fff;
}

.submenu {
  margin-top: 4px;
  padding-left: 12px;
  display: flex;
  flex-direction: column;
  gap: 3px;
}

.submenu-item {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 8px 14px;
  border-radius: 6px;
  cursor: pointer;
  font-size: 0.92rem;
  color: #adb5bd;
}

.submenu-item:hover {
  background-color: rgba(255, 255, 255, 0.08);
  color: #fff;
}

.submenu-item.active {
  background-color: #007bff;
  color: #fff;
  font-weight: 600;
}

.sidebar-mini .sidebar {
  width: 65px;
}

.sidebar-mini .brand-text,
.sidebar-mini .menu-label,
.sidebar-mini .menu-arrow,
.sidebar-mini .submenu {
  display: none !important;
}

.main-wrapper {
  flex: 1;
  display: flex;
  flex-direction: column;
}

.topbar {
  height: 56px;
  background-color: #ffffff;
  border-bottom: 1px solid #dee2e6;
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0 20px;
}
```

```
.topbar-left {
  display: flex;
  align-items: center;
  gap: 15px;
}

.menu-toggle {
  background: none;
  border: none;
  font-size: 1.3rem;
  color: #6c757d;
  cursor: pointer;
}

.top-link {
  text-decoration: none;
  color: #6c757d;
  font-weight: 500;
}

.user-info {
  display: flex;
  align-items: center;
  gap: 10px;
}

.avatar {
  width: 32px;
  height: 32px;
  background-color: #17a2b8;
  color: #fff;
  border-radius: 50%;
  display: flex;
  align-items: center;
  justify-content: center;
  font-weight: bold;
}

.content-area {
  flex: 1;
  padding: 24px;
}

.estado-box {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 320px;
}

.estado-card {
  background-color: #ffffff;
  border: 1px solid #dee2e6;
  border-radius: 12px;
  padding: 40px;
  text-align: center;
  max-width: 500px;
}

.toolbar {
  display: flex;
```

```
  justify-content: space-between;
  align-items: center;
  margin-bottom: 16px;
  background-color: #fff;
  padding: 16px 20px;
  border-radius: 8px;
  border: 1px solid #e3e6f0;
}

.btn-nuevo {
  background-color: #007bff;
  color: #fff;
  border: none;
  padding: 9px 18px;
  border-radius: 6px;
  font-weight: 600;
  cursor: pointer;
}

.tabla-container {
  background-color: #ffffff;
  border-radius: 8px;
  border: 1px solid #e3e6f0;
  overflow: hidden;
}

.tabla {
  width: 100%;
  border-collapse: collapse;
}

.tabla th {
  background-color: #f8f9fc;
  color: #4e73df;
  font-weight: 700;
  padding: 12px 14px;
  border-bottom: 2px solid #e3e6f0;
}

.tabla td {
  padding: 12px 14px;
  border-bottom: 1px solid #f1f3f5;
}

.btn-accion {
  border: none;
  border-radius: 4px;
  padding: 6px 10px;
  margin: 0 3px;
  cursor: pointer;
}

.btn-editar { background-color: #ffc107; color: #000; }
.btn-detalle { background-color: #17a2b8; color: #fff; }
.btn-eliminar { background-color: #dc3545; color: #fff; }

.footer {
  background-color: #ffffff;
  border-top: 1px solid #dee2e6;
  padding: 12px 20px;
  font-size: 0.85rem;
```

```
  color: #6c757d;
  text-align: right;
}

.modal-overlay {
  position: fixed;
  top: 0;
  left: 0;
  width: 100vw;
  height: 100vh;
  background-color: rgba(0, 0, 0, 0.5);
  display: flex;
  align-items: center;
  justify-content: center;
  z-index: 1050;
}

.modal-caja {
  background-color: #fff;
  border-radius: 10px;
  width: 90%;
  max-width: 520px;
  overflow: hidden;
}

.modal-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 16px 20px;
  background-color: #f8f9fa;
  border-bottom: 1px solid #dee2e6;
}

.btn-cerrar {
  background: none;
  border: none;
  font-size: 1.2rem;
  cursor: pointer;
}

.modal-body { padding: 20px; }

.modal-footer {
  display: flex;
  justify-content: flex-end;
  gap: 10px;
  padding-top: 16px;
  border-top: 1px solid #eee;
}

.form-grupo { margin-bottom: 14px; }
.form-grupo label { display: block; font-weight: 600; margin-bottom: 5px; }
.form-input { width: 100%; padding: 8px 12px; border: 1px solid #ced4da; border-radius: 6px; box-
sizing: border-box; }
.form-fila { display: grid; grid-template-columns: 1fr 1fr; gap: 12px; }
.calculos-box { background-color: #e8f4fd; border: 1px solid #b8daff; border-radius: 6px; padding:
10px 14px; display: flex; justify-content: space-between; margin: 10px 0; }
.detalle-lista { list-style: none; padding: 0; margin: 0; }
```

```
.detalle-lista li { padding: 8px 0; border-bottom: 1px solid #f1f3f5; }
.badge-total { font-weight: bold; color: #28a745; font-size: 1.1rem; }
```

---

5. Parte 3: Cronograma Estratégico de 90 Minutos para el Examen

Bloque	Minutos	Qué Hacer	Criterio de Aprobación
1	**00 - 20**	**Backend .NET:** Crear Modelos, DbContext, `DADetalleEjecuciones` y Controller con CORS	Compila en Visual Studio sin errores
2	**20 - 30**	**Swagger:** Ejecutar el backend y probar `GET /api/ejecuciones` y `POST`	Devuelve `200 OK` con JSON
3	**30 - 45**	**Front Cimientos:** `provideHttpClient()` en `app.config.ts`, `detalle.model.ts` y `ejecuciones.services.ts`	Servicio compila sin errores
4	**45 - 75**	**Front Interfaz:** Pegar `panel.ts`, `panel.html` y `panel.css`. Probar que al dar clic en **Ejecuciones** se liste la tabla	Los datos de la base de datos se ven en la tabla
5	**75 - 90**	**Pruebas Finales:** Probar botón **Nuevo** (Crear), **Editar** y **Eliminar**	Flujo completo cerrado y funcionando

---

6. Parte 4: Top 5 Errores Típicos de Examen y Solución en 5 Segundos

1. **Error CORS (`blocked by CORS policy`)**:
2. *Solución:* Agrega `app.UseCors("PermitirTodo");` en `Program.cs` antes de `app.MapControllers()`.
3. **Error `NG02800: provideHttpClient()`**:
4. *Solución:* En `app.config.ts` agrega `provideHttpClient()` en el arreglo `providers`.
5. **Error `Can't bind to 'ngModel' since it isn't a known property`**:
6. *Solución:* En `panel.ts`, agrega `FormsModule` dentro del array `imports: [CommonModule, FormsModule]`.
7. **La tabla se queda en blanco o en error**:
8. *Solución:* Revisa el puerto de tu Backend. Si en tu navegador Swagger está en `http://localhost:5000` o `7000`, actualiza esa misma URL en `ejecuciones.services.ts`.
9. **Carpeta gris con flecha rota en GitHub**:
10. *Solución:* Ocurre cuando Angular CLI crea una carpeta `.git` interna. Ejecuta `Remove-Item -Recurse -Force .\frontend-angular\.git`, haz `git add .` y sube los cambios.